Exposing Cross-Group Parallelism in Layer-Aware Lookahead Construction for 3D FPGA Routing

Anonymous Authors

***Abstract*—Native multicore routing leaves costly lookahead construction insufficiently parallelized. We present a semantics-preserving decomposition for 3D FPGA lookahead: independent layer-aware floods, ordered cost reduction, and cross-group execution with reusable private workspaces. Slice-local interpolation and compact storage further expose scheduling and memory tradeoffs. On eight-core GEMM, construction falls from 149.56 to 20.95 seconds and process speedup is $1.638 \pm 0.005\times$. On two large 3D graphs, Pipeline scales approximately $12\times$ from one to sixteen cores while group-local execution saturates. Native equivalence checks, full-routing experiments, and direct global-task controls establish the benefit of removing group barriers and a verified task boundary for future hardware acceleration.**

***Index Terms*—3D FPGA, routing lookahead, multicore CAD, deterministic reduction, architecture exploration**

I. Introduction

Three-dimensional FPGA integration expands the architecture design space through stacked active fabrics, heterogeneous resources, and programmable vertical interconnects. Evaluating this space requires CAD tools that can both represent these structures and process their resource graphs efficiently. LaZagna demonstrates flexible architecture generation and evaluation, while Escaping Flatland improves placement through layer assignment, delay modeling, and 3D move operators [1], [2]. These advances increase the value of a fast routing backend: more candidate architectures and placements can be assessed within a fixed exploration budget. Changing connectivity or calibrated costs can require a new lookahead table even when the subsequent router already supports multiple CPU cores.

VPR's native parallel router does not make every part of a routing invocation parallel. The map-lookahead construction in our locked baseline enumerates representative wire samples and executes their graph searches sequentially. Increasing the routing worker budget therefore leaves a substantial initialization bottleneck. On a two-layer GEMM resource graph, stock VPR with eight physical cores spends 149.56 ± 1.32 seconds constructing lookahead, compared with 30.64 ± 0.38 seconds in routing, within a 331.05 ± 1.36-second process. Construction accounts for approximately 45% of this process, including loading and checks. Optimizing only the routing search cannot remove that cost.

A direct first step is to execute the samples of each wire group concurrently. However, representative groups often contain only four samples. With a barrier after every group, assigning eight or sixteen workers still exposes at most four simultaneous sample searches. The challenge is to expose parallelism across groups while preserving the numerical model used by timing-driven routing. Equal-delay samples carry associated congestion costs; completion-order reduction can change the selected pair. In 3D, results additionally distinguish source and target layers, vertical wire directions, and missing slices. Naively parallelizing the surrounding loops can introduce write conflicts during interpolation.

We address these issues by separating native semantics from execution order. Each unchanged sample flood operates on private search state and produces a layer-indexed result. An ordered reduction preserves the native delay/congestion winner, while a global task stream removes group barriers. The implementation uses VPR's existing oneTBB worker budget. A bounded-token Pipeline overlaps searches and reductions; a direct global parallel loop provides an independent execution realization. We also analyze slice-local interpolation and compact intermediate storage to expose additional dependencies and resource tradeoffs.

Our contributions are the following.

1) **A semantics-preserving parallel decomposition.** We identify independent sample floods, retain source-relative coordinates and strict-delay tie semantics, and formulate ordered reduction as the correctness boundary between concurrent execution and the native map.
2) **Cross-group execution inside native VPR.** We replace group-limited concurrency with global sample scheduling, reusable private workspaces, and bounded in-flight results. Direct global-loop controls establish which improvements arise from removing group barriers, rather than attributing all gains to a particular library primitive.
3) **Explicit support for layer-aware construction and storage.** We preserve CHANZ sampling, all target-layer slices, native inter-layer cost semantics, and unsampled-layer completion. A slice dependency analysis permits equivalent commit schedules and reduces redundant interpolation scans; compact cells and a budgeted variant quantify the resulting storage/performance tradeoffs.
4) **Native validation from construction to routing workflows.** We evaluate 603 formal measurements across 2D and two-layer 3D inputs, separating 414 legal full-flow runs from 189 construction-only runs. Scaling, same-executable controls, cache sequences, memory diagnostics, and independent route checks connect the algorithmic decomposition to its practical benefit and remaining costs.

Pipeline achieves $1.638 \pm 0.005\times$ paired process speedup on eight-core GEMM. On BNN and DLA 3D graphs, its one-to-

sixteen-core construction speedups are $12.006 \pm 0.167\times$ and $12.253 \pm 0.296\times$, respectively. These results establish effective use of parallelism beyond individual groups. They also expose a concrete acceleration boundary for future GPU or FPGA engines: independent searches may be replaced while retaining an ordered, testable map-construction interface.

II. Background and Problem Formulation

A. 3D Resource Graphs and Lookahead Semantics

An FPGA routing-resource graph is a directed graph $\mathcal{R} = (V, E)$ whose vertices represent wires and pins, with capacity, location, type, and cost information. PathFinder-style negotiation resolves contention between routed nets [3]. VPR combines congestion and timing costs with an estimate of the remaining cost to a sink [4], [5]. The sampled map-lookahead approach constructs these estimates by flooding representative wire starts and recording costs at relative destination coordinates [6].

For a layered architecture, let the wire-cost table be

$$F[\ell_s, \ell_t, c, s, \Delta x, \Delta y] = (d, q), \tag{1}$$

where ℓ_s and ℓ_t are source and target layers, c is channel type, s is segment type, and d, q are the retained delay and congestion estimates. A sampling group $g = (\ell_s, s, c)$ contains an ordered list of starts $R_g = (r_{g,0}, \ldots, r_{g,n_g-1})$. Each sample can reach destinations on every layer. Missing entries are completed using native interpolation. Source/OPIN preprocessing and derived minima are additional parts of the full construction stage.

Our 3D inputs model two active fabric layers containing logic, DSP, and memory resources, connected by explicit vertical segments and inter-layer switches. The architecture specifies `axis=z` segments, upward/downward links, and fan-in-dependent switch delays. Actual routes contain CHANZ transitions between $(x, y, 0)$ and $(x, y, 1)$. Thus the 3D classification follows from architecture, connectivity, and routed paths together, rather than layer labels alone. We adopt the existing VTR multi-die model [7], [8]; our contribution is its parallel construction, not the invention of vertical routing resources.

The native wire flood uses its existing indexed delay and congestion model. We preserve these computations, graph edges, sample choices, and traversal semantics. We do not replace the cost model with Manhattan distance or claim a new per-edge physical-delay model. In particular, the placement-delay refinement in Escaping Flatland [2] is a distinct contribution that motivates, but is not implemented by, this work.

B. Available Threads Versus Available Work

The stock executable already receives the same physical-core budget and enables native parallel routing in our process comparisons. Its wire-sampling loop remains sequential. Our within-group implementation, GROUP, first exposes sample concurrency, but synchronizes before the next group. The cross-group implementations instead expose the union of all independent sample tasks.

Let $t_{g,j}$ denote ideal sample-search work, $W_g = \sum_j t_{g,j}$, and $W = \sum_g W_g$. Ignoring synchronization, memory contention, allocation, and reduction, lower bounds on search time under P workers are

$$T_{\text{group}} \geq \sum_g \max\{W_g/P, \max_j t_{g,j}\}, \tag{2}$$

$$T_{\text{global}} \geq \max\{W/P, \max_{g,j} t_{g,j}\}. \tag{3}$$

The second schedule allows work from one group to occupy workers unavailable to the first. For ten groups of four equal-cost samples, group-local execution requires ten waves at both eight and sixteen cores. Global execution needs five and three waves, respectively. The sixteen-core discrete speedup limit of this example is $40/3 = 13.33$ relative to one core, rather than sixteen. This illustrates available task parallelism; it is not a measured CPU-utilization percentage or a guarantee of optimal scheduling.

C. Related Parallel and 3D CAD Methods

MultiQueue routing exposes parallelism within an individual connection search using relaxed priority ordering [9]. Our unit of concurrency is an unchanged flood: we parallelize independent samples and groups while maintaining each flood's native search order. The two granularities can potentially be combined, but that composition requires separate validation.

VPR offers alternative routing algorithms, compressed lookahead, and table read/write support [10]. Compressed lookahead trades representation and sampling choices against construction cost; our main comparisons preserve the original map. The locked compressed implementation accepts only a single layer, so its rejection of a two-layer input is treated as an applicability limitation. Compatible 2D alternatives are evaluated separately. Global `parallel_for`, private workspaces, and ordered pipelines are existing oneTBB mechanisms [11]; the contribution is the dependency decomposition and equivalent native integration that make those mechanisms useful for layer-aware construction.

LaZagna expands the range of realizable 3D architectures, including vertical connection patterns and heterogeneous layers [1]. Escaping Flatland improves the quality of placements on such architectures [2]. Our work complements these objectives by reducing the cost of constructing an unchanged routing heuristic. A faster backend can support more architecture candidates and repeated placement evaluation without confounding algorithmic quality with a changed heuristic.

III. Semantics-Preserving Cross-Group Construction

A. Independent Floods With Private State

For each native group g, sample selection follows the original reference locations, segment and direction filters, and fallback search for unusual wire types. We retain groups for supported X, Y, and Z channel types and respect layers without inter-cluster programmable resources. A task descriptor identifies its group, sample index, and start RR node. Its output $C_{g,j}[\ell_t, \Delta x, \Delta y]$ includes every destination layer.

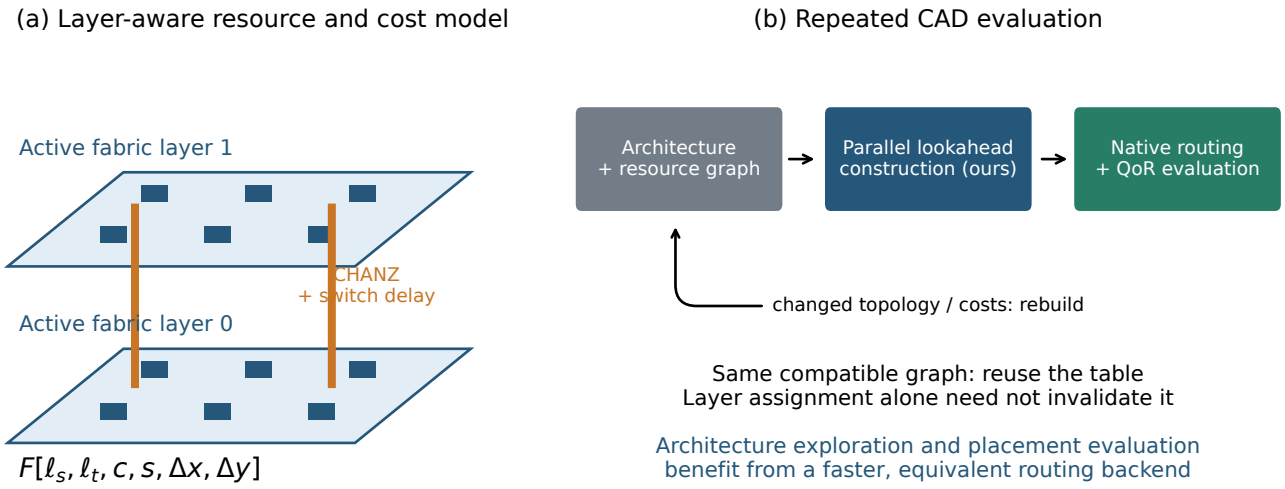


Fig. 1. Research context. (a) Two active fabric layers connect through modeled vertical routing resources; the lookahead retains source/target layer identity. (b) Faster equivalent construction reduces the cost of evaluating changed resource graphs. Reusing a compatible graph permits caching; changing placement alone does not necessarily require reconstruction. The schematic is conceptual, not a physical implementation measurement.

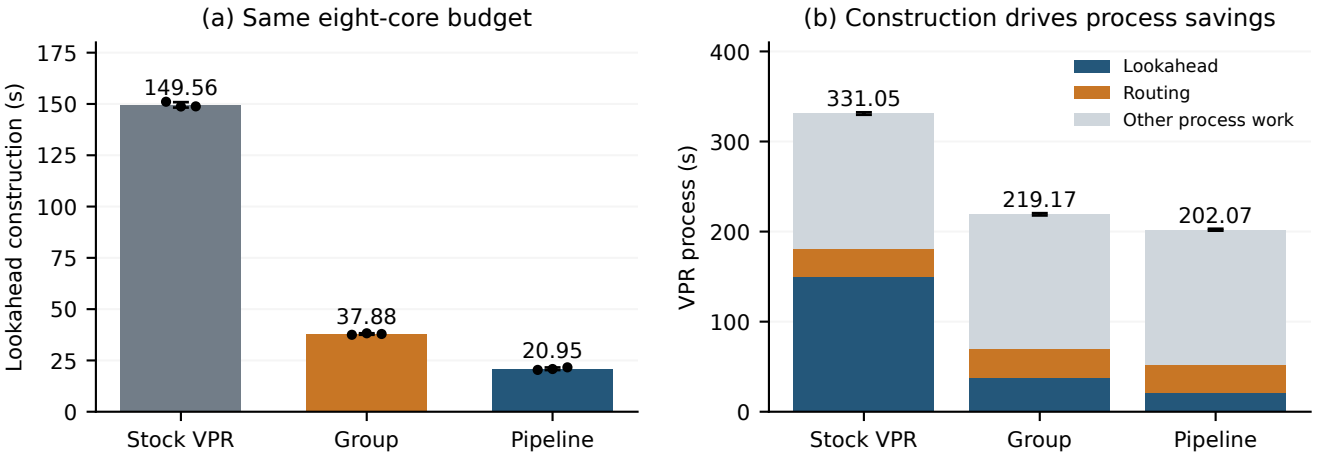


Fig. 2. Measured GEMM motivation at the same eight-physical-core budget, three repeats per implementation. (a) Complete lookahead construction: Stock 149.56 ± 1.32 s, Group 37.88 ± 0.37 s, Pipeline 20.95 ± 0.64 s. (b) Full fixed-input VPR process; the residual includes loading, checking, and all other process work. Bars show means; error bars show sample SD. Stock already enables native parallel routing. Group is our within-group implementation, not an upstream baseline.

Every worker owns an expanded bitmap, visited-cost array, priority queue, and sample output. The resource graph and cost data are read-only during construction. A worker may reuse its allocated workspace for subsequent tasks, but resets search state for every new flood. Reuse of storage therefore does not merge searches or propagate labels from a different source. The original full-device bounding box spans all layers, so paths can traverse native CHANZ resources while retaining layer-dependent costs.

Source identity must survive the decomposition. Consider two starts at $x = 0$ and $x = 5$ reaching a pin at $x = 6$ with delays 1 and 2. The native samples contribute to displacement buckets 6 and 1, respectively. A nearest-source search keeping only the cheaper label at the pin loses the second contribution. Consequently, shared destinations do not justify replacing the samples with a single unlabeled multi-source flood. Even repeated start RR IDs require matching recording context and search constraints before exact reuse is valid. We retain all selected samples.

B. Ordered Reduction and Correctness

The native smallest-entry rule replaces a populated cell only when the new delay is strictly smaller. Its congestion value must remain paired with that delay. For finite valid values, the winner at a cell u is

$$j^*(u) = \arg \min_{j : C_{g,j}(u) \text{ occupied}} (d_{g,j}(u), j). \tag{4}$$

Algorithm 1 Layer-aware cross-group Pipeline construction

Require: RR graph $\mathcal{R}$, native cost data, worker budget P
Ensure: Native-equivalent wire table F
$\mathcal{G} \leftarrow$ native ordered (ℓ_s, s, c) groups
$J \leftarrow [\,]$; initialize native-empty group tables B_g
for $g \in \mathcal{G}$ in native order **do**
  $R_g \leftarrow$ NATIVESELECT$(\mathcal{R}, g)$
  for $j = 0, \ldots, |R_g| - 1$ **do**
    append $(g, j, R_g[j])$ to J
  end for
end for
$Q \leftarrow \min(|J|, 2P)$ in-flight sample tokens
Ordered producer:
Emit descriptors from J in order, subject to Q
Parallel worker for descriptor (g, j, r)*:*
$W_t \leftarrow$ private reusable workspace; reset its labels/queue
$C \leftarrow$ NATIVEFLOOD$(\mathcal{R}, r, W_t)$
Publish (g, j, C) with all target-layer cells
Ordered consumer, in descriptor order:
$B_g \leftarrow$ NATIVESTRICTMERGE(B_g, C)
Release C and its token
After pipeline completion:
for $g \in \mathcal{G}$ in native order **do**
  $F \leftarrow$ NATIVECOMMITANDINTERPOLATE(F, g, B_g)
end for
Run unchanged source/OPIN and derived-minimum construction

The cell contains (d_{g,j^*}, q_{g,j^*}), not two independently minimized scalars. We implement the native insertion and occupancy predicates directly, rather than relying on a generic floating-point tuple minimum.

Proposition 1: Assume native sample selection and within-flood visitation are unchanged, recorded valid delays are finite, and final commits and interpolation follow the native order. Private sample execution followed by original-order strict-delay reduction yields the same wire-cost table as serial sample execution.

Proof. At each cell, one sample's representative is the minimum-delay entry first encountered within that sample. Native serial execution concatenates these per-sample visitation sequences. Reducing their representatives in sample order selects the minimum delay in the concatenation and retains its earliest associated congestion on ties. Thus each populated cell agrees; missing cells retain the same occupancy state. Identical commits and interpolation then produce identical final cells. The argument is independent of task completion order. □

This proposition does not license arbitrary reassociation of exceptional floating-point values. NaNs, invalid sentinels, signed zeros, and occupied-but-invalid states are handled by the native insertion rules and separate representation tests. It also does not require final parallel route files to be byte-identical: routing has its own concurrent state and trajectory.

C. Three Execution Schedules

Group-local execution. GROUP allocates private sample results for one group, executes its floods in parallel, and reduces them in original order. The next group starts after this barrier. This establishes sample independence but limits active work to the current group's sample count.

Direct global execution. FLAT materializes the same ordered descriptors for all groups, executes them with one global `parallel_for`, then reduces all results in descriptor order after the join. Workspace allocation is thread-local and spans groups. This schedule isolates the effect of globally exposing the task set without an ordered pipeline.

Ordered pipeline execution. Algorithm 1 uses an ordered producer, parallel flood body, and ordered consumer. At most $\min(S, 2P)$ sample results are in flight for S total samples. Later groups can execute while an earlier group is still outstanding. Results are reduced and released incrementally; commits retain native group order. Head-of-line waiting at the consumer and per-flood imbalance can still limit speed. FLAT and PIPELINE share task enumeration, flood code, result representation, and native commit order in their direct comparison.

The global implementations also change allocation lifetimes and search/reduction overlap. We therefore interpret Flat-to-Pipeline measurements as a comparison of complete execution realizations, not a measurement of scheduling overhead alone. The TBB budget limits allowed parallelism, while observed workspace count is measured separately; it is not assumed to equal the thread budget in every run.

D. What the Algorithm Preserves for 3D FPGA Routing

The layer dimension affects the algorithm at several specific points.

1) **Layer- and direction-aware task enumeration.** The task key retains source layer, segment, and channel type, including CHANZ. Native vertical direction selection and fallback behavior are preserved. These are inherited graph semantics explicitly maintained by the parallel implementation.
2) **Cross-layer search and destination recording.** Each flood uses the complete multilayer graph. Its result is indexed by target layer as well as displacement, so equal $(\Delta x, \Delta y)$ on different layers cannot alias. Layer crossing is represented by native resources and costs, not an assumed constant geometric penalty.
3) **Layer-pair-specific reduction and completion.** Private results merge only into the matching source-layer group, retaining all target layers. Interpolation operates within fixed layer-pair slices, with native completion for unsampled or disabled source layers.
4) **Resource scaling with layers.** Private maps contain LWH cells and the final map contains layer-pair slices. Token limits and the optional compact representation make this cost explicit instead of silently replicating the entire global map per worker.

The current native CHANZ selection path includes assumptions specific to the tested one-/two-layer configurations. Synthetic higher-layer interpolation tests do not establish a native arbitrary-layer implementation. The algorithmic treatment is layer-aware, but performance portability to other layer counts and vertical fabrics remains a separate evaluation.

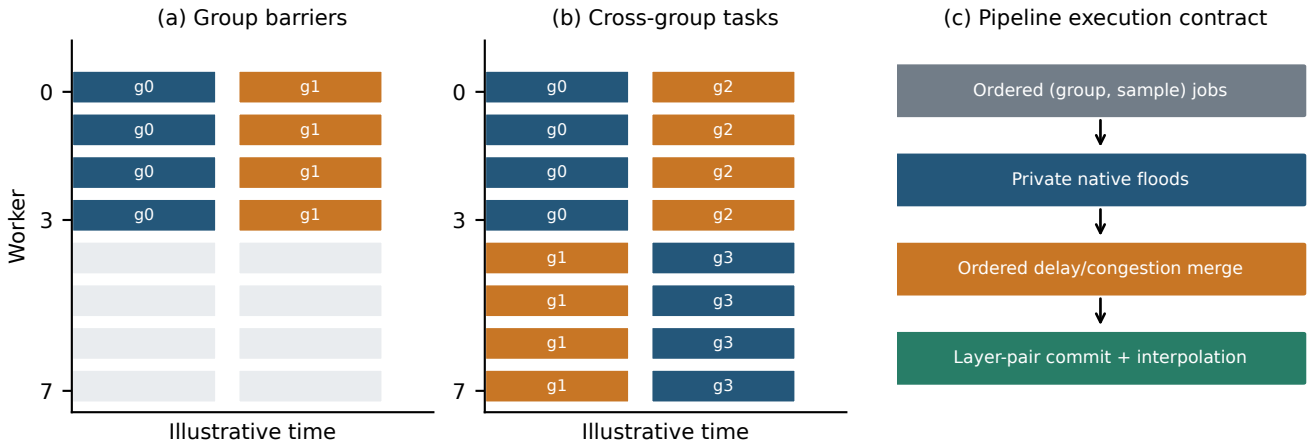


Fig. 3. From local concurrency to global task parallelism. The equal-length schedule is illustrative: a four-sample group cannot occupy eight workers, whereas samples from successive groups can. The actual implementation preserves the ordered map-reduction boundary while allowing independent floods to complete out of order.

IV. Slice Dependencies and Resource-Aware Variants

A. Equivalent Slice Completion

An interpolation call may traverse multiple layers without creating a numerical dependence between them. Inspection of both native neighbor helpers shows that (ℓ_s, ℓ_t, c, s) stays fixed: only $(\Delta x, \Delta y)$ varies. Thus distinct layer-pair slices are independent, while entries within a slice can depend on earlier filled entries. The original outer-x, inner-y scan and floating-point accumulation order must remain unchanged.

The commit for a group overwrites its complete source-layer region, including initially invalid entries. Earlier provisional interpolation in that region is therefore superseded. Once a completed slice is filled, subsequent native calls retain valid entries. Under unique group keys, full-region commits, and no partial-map consumer, these properties permit two equivalent schedules: defer interpolation until all raw groups have been committed, then fill each used channel/segment combination once; or commit and fill only each completed group's source-layer slices, followed by defaults for unsampled layers. Algorithm 2 gives the latter.

Let L be the number of layers, K the used channel/segment combinations, and $A = WH$. If every layer supplies every group, repeated whole-table traversal performs KL^3A missing-cell checks. Completing each final slice once requires KL^2A. This reduces redundant interpolation scans; it does not reduce the number of floods or imply an L-fold overall speedup. Its additional value is enabling early release of completed groups.

B. Compact Cells and Bounded Active Results

In native smallest-entry mode, an intermediate cell needs one occupied flag and the retained (d, q) pair. We implement a scalar cell preserving the vector-based insertion semantics, including the distinction between occupancy and numerical validity. On the measured ABI, a vector header occupies 24 bytes before any element allocation, while the scalar cell occupies 12 bytes. This reduction concerns the intermediate representation, not the entire process RSS.

Algorithm 2 Equivalent source-layer slice completion

Require: Completed group table B_g, $g = (\ell_s, s, c)$
Commit every raw cell of B_g to $F[\ell_s, :, c, s, :, :]$
for every target layer ℓ_t **do**
  $U \leftarrow F[\ell_s, \ell_t, c, s, :, :]$
  for x in native outer-loop order **do**
    for y in native inner-loop order **do**
      if $U[x, y]$ requires completion **then**
        Apply native neighbor rule in native summation order
      end if
    end for
  end for
end for
Publish group completion; release its intermediates
After all sampled groups complete:
for each used (s, c) and unsampled source layer ℓ_s **do**
  Apply the same native slice completion for every ℓ_t
end for
Leave entirely unused (s, c) combinations untouched

The Bounded variant allocates a bounded set of private sample tables and search workspaces. When a group's final sample completes, its worker performs the original-order reduction into the first table, commits and fills that group's slices, and releases the remaining tables. An acquire/release completion counter publishes preceding writes. The implementation uses bounded batches with barriers; a group exceeding the table budget falls back to serial insertion into one table. Slice-only, compact-cell, and bounded variants are evaluated separately from the primary Pipeline.

For $M = LWH$, G allocated group tables, and S total samples, the table-count relationships are summarized in Table I. Group metadata and task descriptors add $O(G + S)$ storage. Search labels, priority queues, allocator state, the resource graph, and the final map are additional. A workspace estimate cannot provide a hard RSS limit because queues and allocator

TABLE I
INTERMEDIATE-TABLE STRUCTURE. COUNTS EXCLUDE THE COMMON FINAL MAP AND RESOURCE GRAPH; VECTOR PAYLOADS AND SEARCH WORKSPACES ARE ADDITIONAL. S_g IS THE CURRENT GROUP'S SAMPLE COUNT; K_a IS THE BOUNDED ACTIVE-SLOT COUNT.

Method	Sample tables	Group tables	Cell backing
Group	S_g	1	$24M(S_g+1)$
Flat	S	G	$24M(S+G)$
Pipeline	$\min(S,2P)$	G	$24M[G+\min(S,2P)]$
Bounded	K_a	in-place	$12MK_a$

caches can grow.

C. Native Integration and Layered Validation

The implementation operates inside native VPR rather than replacing its router with a graph-only proxy. Two bounding-box notification repairs are shared by the CPU variants; these maintain parallel-router state but can change routing trajectories relative to Stock. We therefore distinguish Stock comparisons from same-repair, same-executable method comparisons. The repairs are integration prerequisites, not claimed first discoveries of those upstream issues.

Validation has complementary layers. Ordered-reduction enumeration covers 21,825 permutations. Extracted native interpolation tests cover 2,400 sparse fixtures over 1, 2, 3, 4, and 8 layers, including unsampled slices; two equivalent reorderings yield 4,800 bitwise comparisons with zero differences. Mutations that exchange the x/y scan or omit unsampled-layer completion fail 1,905 and 1,902 fixtures, respectively. Scalar-cell tests cover 100,000 fixtures and 1,551,076 bitwise field comparisons, including exceptional values. Finally, native runs compare serialized wire maps and, for complete routes, apply native FULL checking and independent original-VPR FULL/STA verification. Wire-map equality does not assert equality of every derived source/OPIN structure or final route trajectory.

V. EXPERIMENTAL METHODOLOGY

A. Input Workloads and 3D Classification

Table II lists the inputs used in the main performance studies. They combine neural-network compute circuits, DSP/arithmetic designs, and traditional logic benchmarks. A case consists of architecture XML, BLIF, a packed netlist, fixed placement, and frozen RR graph. Input matrices or neural-network training samples are not supplied to the router; these are hardware netlists mapped onto the listed fabrics.

The KOIOS-derived 3D model and Hecate 3D model contain two active layers and explicit CHANZ resources. The native k6 architecture and original KOIOS 2D architecture are single-layer controls without CHANZ nodes. BNN appears on both a 3D and a separately prepared 2D fabric; this tests applicability, not a matched physical-quality comparison between 2D and 3D. Different placement seeds on a fixed architecture can share a resource graph and are not counted as independent graph topologies.

B. Cohorts and Experimental Boundaries

The evidence comprises 603 formal records: 414 legal full-flow runs and 189 construction-only measurements. Table III separates the cohorts so neither repeated controls nor successful subsets inflate the workload count. The original 222 full-flow records contain GEMM timing/scaling, fresh-layout studies, three-circuit confirmation, and storage ablation. New complete-flow cohorts add a three-layout 3D subset, four k6 layouts, clma, and BNN-2D. Separate construction-only cohorts include all three large 3D graphs, regardless of their route outcome.

The three-layout width-120 full-flow subset was selected by a declared baseline qualification: LSTM seed 2 and both BNN seeds. The original six-layout set retains three successes and three failures; DLA seeds 1/2 and LSTM seed 1 are not silently replaced. All three graphs remain in the graph-only construction study. Construction experiments stop native routing after one iteration; their valid construction timers and maps do not establish legal complete routes. Pilots, 36 new full-flow qualification runs, 18 instrumented diagnostics, six compressed-map alternatives, and 90 cache-batch processes are reported outside the 603 formal records.

C. Baselines, Platform, and Timing

The baseline is VTR commit `0b912334fdd8`, with GNU C++ 11.4 release builds and assertion level 2. Stock uses the original executable. Fixed-serial retains the same CPU-variant state repairs with sample parallelism disabled; Group enables per-group sample parallelism; Flat and Pipeline expose all groups. All full-flow arms use native parallel routing, the same worker budget, the same fixed inputs, and comparable map-output settings. Flat here denotes globally flattened *lookahead tasks*, not VPR's separate flat-routing mode, which is disabled.

Measurements use Intel Xeon Gold 6530 CPU instances with one logical CPU per selected physical core and affinity within one NUMA node. Original/width-120/k6 measurements use the original instance; scaling and flat-control cohorts use additional shared instances. We retain the actual command, environment, topology, affinity, executable hash, and input hashes. Cohorts on different instances are never paired or pooled. Physical-core exclusivity is not established.

Construction time T_{LA} is the complete native lookahead timer: wire sampling, interpolation, source/OPIN work, and derived minima, excluding RR loading and map serialization. Process time T_{proc} includes fixed-input loading, lookahead, routing, native checking/STA, and uniform map output. Workflow time additionally includes independent original-VPR readback/checking. These process measurements do not execute new synthesis, packing, placement, or RR generation. Performance runs disable diagnostic counters. A shared-cache comparison uses the same qualified wire map; native source/OPIN work may still execute.

TABLE II

DATASET INVENTORY. GRID DIMENSIONS ARE $X \times Y \times L$; $L = 2$ ROWS ARE 3D, $L = 1$ ROWS ARE 2D. NODES AND EDGES ARE IN MILLIONS; CHANZ COUNTS ARE EXACT. NETS EXCLUDE GLOBAL/UNROUTED-EMPTY NETS. BLOCKS ARE PLACED CLUSTERED BLOCKS. WIDTH IS 100 FOR THE FIRST SEVEN ROWS AND NATIVE K6 CASES, 120 FOR BNN-3D/LSTM/DLA AND BNN-2D. THE LATTER 3D GRAPH COUNTS ARE TAKEN FROM THE ACTUAL WIDTH-120 XML. TWO LAYOUTS OF A CIRCUIT SHARE THE REPORTED SIZE.

Circuit	Workload type	Fabric	Grid	Layouts	Blocks	Nets	Nodes (M)	Edges (M)	CHANZ
gemm	Matrix multiply	KOIOS-3D	176×176×2	1	38,568	206,155	9.596	84.214	185,856
softmax	NN softmax	KOIOS-3D	42×42×2	1	2,544	11,320	0.527	4.590	10,584
reduction	NN reduction	KOIOS-3D	102×102×2	1	12,380	50,369	3.224	28.136	62,424
elliptic	DSP filter	Hecate	30×30×2	1	506	2,849	0.265	2.366	6,658
alu4	Logic / ALU	Hecate	10×10×2	2	133	713	0.022	0.197	576
diffeq	Arithmetic	Hecate	10×10×2	2	206	1,084	0.022	0.197	576
s38417	Sequential logic	Hecate	20×20×2	2	556	4,444	0.121	1.019	2,936
BNN-3D	Binary NN	KOIOS-3D	112×112×2	2	15,331	78,308	4.103	39.481	75,264
LSTM	Recurrent NN	KOIOS-3D	113×113×2	2	17,432	105,855	4.167	40.142	76,614
DLA	DL accelerator	KOIOS-3D	154×154×2	2	28,507	154,021	7.752	74.858	142,296
tseng	Arithmetic	k6	30×30×1	2	284	572	0.093	0.860	0
des	Cryptographic	k6	40×40×1	2	680	954	0.167	1.570	0
clma	Logic	k6	44×44×1	1	1,080	4,585	0.203	1.918	0
BNN-2D	Binary NN	KOIOS-2D	177×177×1	1	15,331	78,308	5.018	44.368	0

TABLE III

FORMAL EVIDENCE COHORTS. RUNTIME REPEATS ARE WITHIN FIXED CONFIGURATIONS, NORMALLY $n = 3$; FRESH LAYOUTS HAVE ONE TIMING OBSERVATION PER CONFIGURATION. HOST LABELS DISTINGUISH EXECUTION COHORTS, NOT RESOURCE EXCLUSIVITY.

Cohort	Scope	Runs	Host	Design and purpose
Original three-way/extension	Full flow	66	A	GEMM: three variants and 4/8/16-core extension; alu4/diffeq/s38417: two fresh layouts each.
Independent confirmation	Full flow	36	A	GEMM, softmax, reduction; Group/Pipeline; rebuild/cache.
Representation ablation	Full flow	120	A	GEMM/elliptic; five implementations; 4/8 cores; rebuild/cache.
Large 3D construction	Construction	36	A	BNN/LSTM/DLA; Stock, fixed-serial, Group, Pipeline; eight cores.
Qualified large 3D subset	Full flow	72	A	Three layouts, two circuits; four arms; rebuild/cache; eight cores.
Independent k6 architecture	Full flow	96	A	tseng/des, two layouts each; four arms; rebuild/cache.
Core scaling	Construction	90	B	BNN/DLA/tseng; Group/Pipeline; 1/2/4/8/16 cores.
Direct global-loop controls	Construction	54	C	BNN/DLA/clma; Group/Flat/Pipeline; 8/16 cores.
clma complete routing	Full flow	12	C	One layout; Stock/Group/Flat/Pipeline; eight cores.
Original KOIOS 2D BNN	Both	21	C	Twelve full-flow and nine construction runs; eight cores.

D. Statistics, Quality, and Audit

For each fixed configuration, times and paired ratios are reported as arithmetic mean ± sample SD. For repeat r,

$$s_r = \frac{T_{\mathrm{control},r}}{T_{\mathrm{candidate},r}}, \quad \bar{s} = \frac{1}{n}\sum_r s_r, \quad \sigma_s = \sqrt{\frac{\sum_r (s_r - \bar{s})^2}{n-1}}. \tag{5}$$

We compute ratios before averaging; dividing mean times is a different statistic. SD describes the observed repetition variability and is not a confidence interval. Single-observation configurations have no estimated SD. Figures show mean/SD and, where space permits, individual observations. All raw measurements are retained in the artifact.

Cross-circuit summaries first take the geometric mean of per-layout mean paired ratios within each circuit, then the geometric mean across circuits. This gives equal circuit weights; BNN's two layouts do not receive twice LSTM's weight. Separate cache modes, timing scopes, and host cohorts are not combined. This manuscript consistently recomputes mean/SD summaries from frozen observations; original experimental protocols and selection records are retained unchanged.

Full-flow acceptance requires zero overuse, native and independent FULL/STA checks, preserved route identity through independent readback, and CPD and wirelength each within $+5\%$ of the paired control. Map-equivalent comparisons additionally require the reference wire-map bytes. Construction-only validity requires a complete positive build timer before routing, reference-map equality, and the prescribed capped exit. Failed qualifications are retained in their attempted denominators. These distinctions prevent construction success from being interpreted as routing success.

VI. RESULTS

A. From Native Parallel Routing to Parallel Construction

Figure 2 shows the distinction between requesting multiple routing workers and exposing construction work. On the same eight-core GEMM input, mean construction time falls from 149.56 s in Stock to 37.88 s in Group and 20.95 s in Pipeline. The paired Stock-to-Pipeline construction speedup is $7.144 \pm 0.266\times$, and its process speedup is $1.638 \pm 0.005\times$. This is an observed bottleneck in the locked native flow, not a claim that every VPR routing algorithm has poor scaling on every design.

The construction-only study repeats the comparison across all three large 3D graphs (Table IV), including those whose full routes fail qualification. Same-executable fixed-serial controls distinguish the sample-parallel contribution from the integration repairs. Group reduces construction substantially; Pipeline further improves it by approximately 1.9–$2.0\times$ at eight cores. Serial and Group results demonstrate why the cross-group step matters beyond simply enabling a parallel option.

B. Scaling Beyond the Group Width

Figure 4 reports a separate 1/2/4/8/16-core construction cohort on three graphs. BNN and DLA have ten nonempty groups with four samples each. Their Group construction time saturates after four cores, while Pipeline continues improving through sixteen. Relative to its own one-core implementation, Pipeline obtains $12.006 \pm 0.167\times$ on BNN and $12.253 \pm 0.296\times$ on DLA. At sixteen cores, its gains over Group are $3.123 \pm 0.033\times$ and $3.123 \pm 0.001\times$. This directly supports removing the group-local parallel-width limit.

The benefit depends on task availability and cost. The tseng 2D input has only eight samples, so sixteen workers cannot expose sixteen independent floods. Its subsecond stage is also sensitive to timer granularity. DLA's four-core Group-to-Pipeline ratio is $0.954 \pm 0.003\times$, showing that cross-group overhead can outweigh its benefit when a group already supplies enough work. Approximately twelvefold sixteen-core scaling therefore establishes substantial additional concurrency, not perfect utilization or a universal speedup guarantee.

C. Direct Global-Loop Controls

Table V compares Group, Flat, and Pipeline using one executable and the same task-generation/flood code. Both global implementations substantially outperform Group on large 3D graphs at eight and sixteen cores. Their mutual differences are much smaller: the BNN/DLA Flat-to-Pipeline mean ratios range from 0.995 to 1.029. Thus the dominant result is the benefit of exposing all independent groups. Pipeline's incremental release and bounded in-flight sample tables provide a distinct implementation tradeoff, but the current evidence does not establish that it is universally faster than direct global execution.

The larger 2D BNN also benefits: Group construction takes 7.75 ± 0.27 s, compared with 4.23 ± 0.13 s for Flat and 4.16 ± 0.10 s for Pipeline. The tiny clma stage has greater relative variability; retaining all observations prevents an apparently large ratio from being interpreted as a stable scheduling effect.

D. Legal Full-Flow Benefit and Quality

Table VI combines absolute process times and paired speedups, retaining the cohort and cache distinction. On the declared 3D conditional subset, Stock-to-Pipeline mean ratios are 1.638, 1.702, and 1.747 for LSTM seed 2 and BNN seeds 1/2. The equal-circuit geometric aggregate is $1.681\times$; the Group-to-Pipeline aggregate is $1.090\times$. All nine rebuild pairs improve over both Stock and Group. The original three failed layouts remain outside this conditional full-flow claim and inside the broader construction evaluation.

For the 72 formal conditional-subset runs, all native and independent legality checks pass. Pipeline's largest CPD increase relative to Stock is 0%, with maximum wirelength increase about 0.091%. Relative to Group, the largest CPD and wirelength increases are about 0.165% and 0.041%. These changes satisfy the quality gate while showing that equal wire maps do not force identical parallel routing trajectories. Stock includes the state-repair difference; the same-repair controls and phase/cache measurements provide more specific attribution.

The original independent confirmation also retains nine improving rebuild pairs on GEMM, softmax, and reduction. Group-to-Pipeline process speedups are $1.091 \pm 0.008\times$, $1.121 \pm 0.030\times$, and $1.092 \pm 0.008\times$. The new k6 architecture yields an equal-circuit Stock-to-Pipeline rebuild aggregate of $1.129\times$; individual small-design timings have greater relative uncertainty. On the original KOIOS 2D BNN, the full-process Stock-to-Pipeline result is $1.356 \pm 0.028\times$. Its Flat-to-Pipeline process ratio is $1.028 \pm 0.024\times$, whereas the construction ratio within those same full runs is $1.002 \pm 0.044\times$. A small process difference therefore cannot automatically be assigned entirely to construction scheduling.

E. Cache Reuse and Repeated Evaluation

A cache hit bypasses the wire construction being optimized. GEMM's common-cache Stock-to-Pipeline result is $1.000 \pm 0.009\times$, consistent with removal of the measured bottleneck. On the 3D conditional subset, equal-circuit cache aggregates are $1.012\times$ versus Stock and $0.996\times$ versus Group. These controls support construction as the main source of rebuild benefit; a speedup on a bypassed stage is not a requirement of the method.

Actual cache sequences on tseng and des evaluate three repeated batches per arm. Each batch first builds and writes its own equivalent table, then performs four requests using that table. For prefix length K, the measured batch time is

$$B_a(K) = T^{(1)}_{a,\mathrm{build}} + \sum_{k=2}^{K} T^{(k)}_{a,\mathrm{reuse}}. \tag{6}$$

Table VII reports paired ratios of these sums. The $K = 1, 2, 5$ prefixes share observations and are not independent experiments; the five requests use the same fixed design. These measured sequences demonstrate amortization without assuming that every placement change requires rebuilding a compatible table.

F. Storage, Memory, and Representation Ablation

Table VIII measures approximate stage RSS on instrumented eight-core runs after RR loading. Global scheduling increases workspace count from four to eight on the larger inputs, making additional concurrency a real memory tradeoff. For BNN-3D, observed growth is 162.0 MiB for Group, 231.7 MiB for Pipeline, and 241.1 MiB for Flat. Pipeline's sample-table bound is sixteen rather than Flat's forty. On BNN-2D,

TABLE IV

COMPLETE LOOKAHEAD CONSTRUCTION ON ALL THREE WIDTH-120 3D GRAPHS AT EIGHT PHYSICAL CORES. TIMES ARE SECONDS; ENTRIES ARE MEAN ± SAMPLE SD ($n = 3$). RATIO COLUMNS ARE PAIRED CONSTRUCTION SPEEDUPS. THE ONE-ITERATION ROUTE TAILS ARE NOT COUNTED AS LEGAL FULL ROUTES.

Graph	Stock	Fixed-serial	Group	Pipeline	Stock/Pipeline	Group/Pipeline
bnn	66.05 ± 1.07	67.25 ± 0.20	17.44 ± 0.13	8.83 ± 0.29	7.480 ± 0.164	1.976 ± 0.072
lstm	65.85 ± 0.19	67.31 ± 0.91	17.29 ± 0.05	8.87 ± 0.09	7.427 ± 0.061	1.951 ± 0.022
DLA	130.74 ± 1.77	131.71 ± 1.20	33.69 ± 0.36	17.88 ± 0.29	7.316 ± 0.209	1.885 ± 0.016

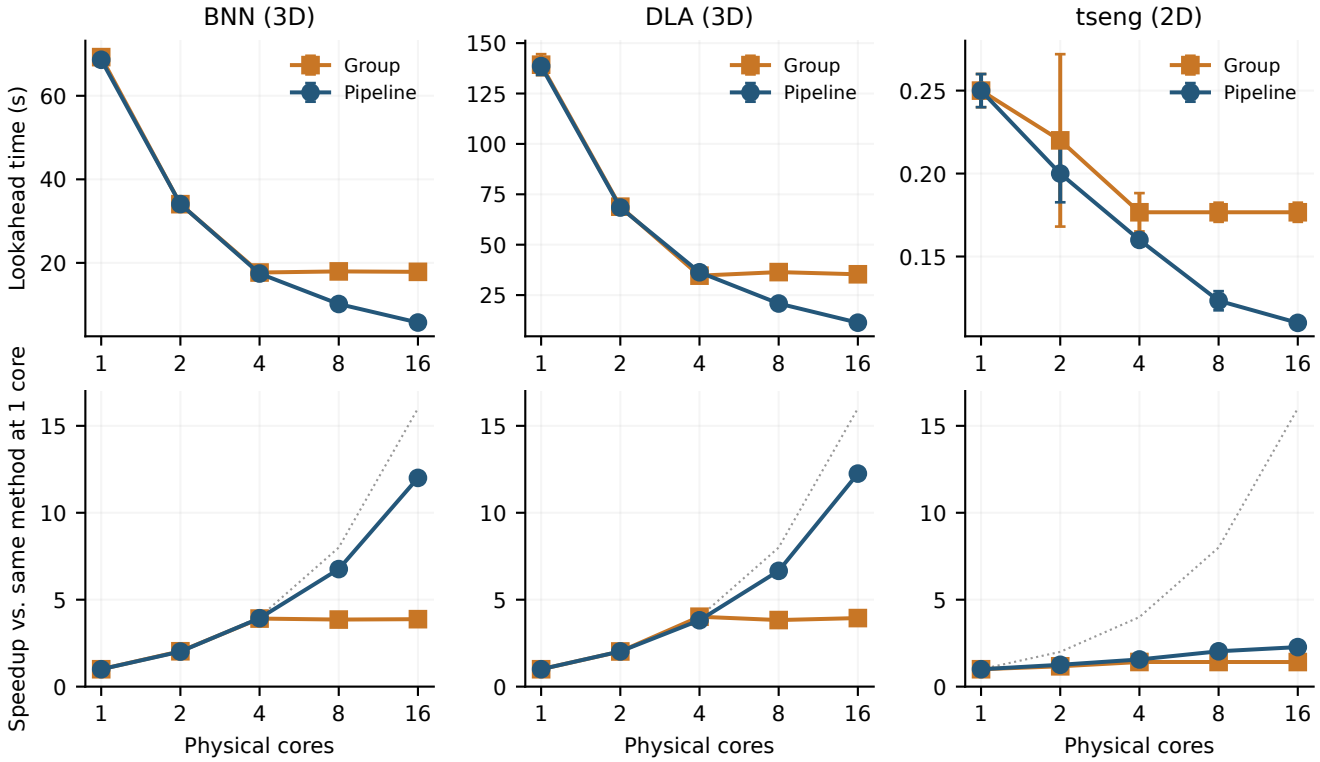


Fig. 4. Construction scaling on host B, with three repeats at each core count. Top: mean construction wall time and sample SD. Bottom: paired speedup relative to the same method at one core; the dotted line is ideal linear scaling. Group saturates near the four-sample group width on BNN/DLA, while Pipeline exposes forty samples globally. tseng has only eight tasks. No process speedup or legal-route claim is derived from this capped-routing cohort.

both global implementations have sixteen-sample bounds and nearly equal observed peaks.

The sampler targets 2 ms but actual gaps can be much longer; observations can miss peaks. Earlier whole-process high-water marks were dominated by RR parsing and could not isolate construction growth. The newer measurements address that ambiguity, while remaining insufficient to claim a stable process-memory saving for Pipeline. The bounded representation limits a different object set and is not validated by subtracting these RSS samples.

Table IX retains the full storage/scheduling ablation. Compact cells and equivalent slice completion pass semantic checks, but the budgeted combination does not consistently improve process time over Pipeline. For eight-core GEMM, Bounded achieves $1.057 \pm 0.046\times$ relative to Group, versus $1.093 \pm 0.003\times$ for Pipeline; the paired Pipeline-time/Bounded-time ratio is $0.968 \pm 0.042\times$. The dependency and storage results remain useful methodological contributions, while Pipeline is retained as the main performance implementation.

G. Alternative Models and Retained Failures

The compatible compressed-map control is evaluated on tseng/des with its own quality gate rather than byte equality to the original map. Its des CPD changes by approximately $+3.95\%$, illustrating that alternative-model comparisons involve quality as well as runtime. It is not combined with the map-equivalent speedup aggregate. The locked compressed implementation's multilayer assertion is retained as an incompatibility, not counted as a victory for our method.

Earlier width-100 qualification attempted thirty runs across five variants and six large-design layouts: seventeen timed out, three stock runs crashed, five failed to converge, and five BNN-seed-2 runs were legal. The original formal matrix was

TABLE V

DIRECT GLOBAL-TASK CONTROLS ON HOST C. ENTRIES ARE MEAN ± SAMPLE SD ($n = 3$); TIMES ARE COMPLETE CONSTRUCTION SECONDS, RATIOS ARE PAIRED STAGE SPEEDUPS. FLAT IS GLOBAL `PARALLEL_FOR` WITH ORIGINAL-ORDER REDUCTION. BNN AND DLA ROWS WITHOUT A 2D SUFFIX ARE 3D.

Graph	Cores	Group	Flat	Pipeline	Group/Flat	Flat/Pipeline
bnn	8	17.41 ± 0.33	9.32 ± 0.15	9.36 ± 0.11	1.869 ± 0.061	0.995 ± 0.010
bnn	16	17.68 ± 0.33	5.51 ± 0.03	5.39 ± 0.08	3.211 ± 0.049	1.021 ± 0.012
DLA	8	34.82 ± 0.32	19.55 ± 0.62	18.99 ± 0.12	1.782 ± 0.060	1.029 ± 0.031
DLA	16	33.99 ± 0.66	10.90 ± 0.04	10.85 ± 0.04	3.119 ± 0.072	1.005 ± 0.006
clma_2d	8	0.35 ± 0.08	0.25 ± 0.07	0.21 ± 0.01	1.516 ± 0.637	1.167 ± 0.293
clma_2d	16	0.31 ± 0.01	0.21 ± 0.01	0.21 ± 0.00	1.469 ± 0.012	1.016 ± 0.027
bnn_2d	8	7.75 ± 0.27	4.23 ± 0.13	4.16 ± 0.10	1.836 ± 0.108	1.017 ± 0.007

TABLE VI

LEGAL FULL-PROCESS RESULTS AT EIGHT PHYSICAL CORES, MEAN ± SAMPLE SD ($n = 3$). B: REBUILD; C: COMMON CACHE. TIMES ARE SECONDS; FINAL COLUMNS ARE PAIRED PROCESS SPEEDUPS. GEMM AND THE LARGE-3D/K6 ROWS ARE SEPARATE HOST-A COHORTS; CLMA/BNN-2D ARE HOST C. FIXED-S IS THE REPAIRED SERIAL-CONSTRUCTION CONTROL. "–" DENOTES AN ARM NOT PRESENT IN THAT COHORT. PREPARATION AND INDEPENDENT-CHECK TIME ARE EXCLUDED FROM THESE PROCESS TIMES.

Case	Mode	Stock	Fixed-S	Group	Pipeline	Flat	Stock/Pipeline	Group/Pipeline
gemm	B	331.05 ± 1.36	–	219.17 ± 1.26	202.07 ± 1.06	–	1.638 ± 0.005	1.085 ± 0.001
gemm	C	181.48 ± 1.47	–	184.43 ± 4.76	181.46 ± 0.29	–	1.000 ± 0.009	1.016 ± 0.027
lstm_s2	B	148.96 ± 1.10	149.08 ± 1.17	97.42 ± 0.71	90.97 ± 1.89	–	1.638 ± 0.041	1.071 ± 0.014
lstm_s2	C	81.60 ± 0.80	81.14 ± 0.97	80.63 ± 0.56	81.56 ± 1.03	–	1.001 ± 0.022	0.989 ± 0.009
bnn_s1	B	138.62 ± 0.33	139.08 ± 0.60	90.53 ± 0.56	81.45 ± 0.54	–	1.702 ± 0.011	1.112 ± 0.007
bnn_s1	C	73.76 ± 1.18	73.23 ± 0.44	72.70 ± 0.55	72.71 ± 0.39	–	1.014 ± 0.013	1.000 ± 0.010
bnn_s2	B	142.87 ± 2.80	140.29 ± 0.57	90.51 ± 0.54	81.78 ± 0.30	–	1.747 ± 0.028	1.107 ± 0.009
bnn_s2	C	75.40 ± 0.62	72.73 ± 0.51	73.32 ± 0.68	72.92 ± 0.50	–	1.034 ± 0.012	1.006 ± 0.011
tseng_s1	B	1.48 ± 0.14	1.42 ± 0.13	1.33 ± 0.03	1.28 ± 0.03	–	1.155 ± 0.104	1.039 ± 0.039
tseng_s1	C	1.25 ± 0.19	1.10 ± 0.03	1.12 ± 0.00	1.13 ± 0.08	–	1.108 ± 0.239	0.988 ± 0.065
tseng_s2	B	1.42 ± 0.05	1.38 ± 0.03	1.33 ± 0.03	1.32 ± 0.00	–	1.076 ± 0.038	1.012 ± 0.021
tseng_s2	C	1.15 ± 0.03	1.25 ± 0.12	1.13 ± 0.03	1.18 ± 0.08	–	0.974 ± 0.064	0.959 ± 0.040
des_s1	B	2.22 ± 0.09	2.22 ± 0.05	2.19 ± 0.08	1.95 ± 0.03	–	1.137 ± 0.039	1.120 ± 0.055
des_s1	C	1.79 ± 0.03	1.82 ± 0.10	1.79 ± 0.03	1.79 ± 0.06	–	1.000 ± 0.028	1.002 ± 0.029
des_s2	B	2.57 ± 0.09	2.59 ± 0.08	2.34 ± 0.03	2.24 ± 0.06	–	1.149 ± 0.009	1.045 ± 0.040
des_s2	C	2.09 ± 0.03	2.07 ± 0.05	2.07 ± 0.05	2.20 ± 0.08	–	0.948 ± 0.047	0.940 ± 0.034
clma_2d	B	5.53 ± 0.09	–	5.22 ± 0.22	5.05 ± 0.10	5.02 ± 0.13	1.095 ± 0.027	1.034 ± 0.042
bnn_2d	B	104.91 ± 2.55	–	81.46 ± 0.77	77.36 ± 0.53	79.51 ± 2.33	1.356 ± 0.028	1.053 ± 0.016

TABLE VII

MEASURED CACHE-SEQUENCE SPEEDUPS, CONTROL/PIPELINE, MEAN ± SAMPLE SD OVER THREE BATCHES. PREFIXES ARE CORRELATED.

Case	Control	$K = 1$	$K = 2$	$K = 5$
tseng_s1	Stock	1.107 ± 0.061	1.057 ± 0.054	0.996 ± 0.055
tseng_s1	Group	1.041 ± 0.041	1.035 ± 0.012	0.979 ± 0.038
des_s1	Stock	1.127 ± 0.028	1.098 ± 0.014	1.067 ± 0.032
des_s1	Group	1.042 ± 0.038	1.018 ± 0.015	1.015 ± 0.017

not launched. Width-120 qualification retained three legal and three failed layouts. These results motivated separate all-graph construction and qualified full-flow studies, not selective removal of unfavorable performance observations. Fresh-layout small-design measurements and low-core regressions are likewise retained. The raw artifact distinguishes qualifications, diagnostics, capped routes, and complete formal runs.

VII. IMPLICATIONS FOR 3D CAD AND HARDWARE ACCELERATION

A. A Faster Backend for Architecture and Placement Exploration

LaZagna's customizable vertical connectivity and heterogeneous layer organization increase the number of meaningful architecture candidates [1]. Each changed graph or cost model can require rebuilding lookahead. Our result reduces this repeated precomputation while preserving the baseline model, so architecture-quality comparisons can retain the same heuristic definition across CPU schedules. Cache reuse remains the preferred option when the graph and relevant cost state are compatible.

Escaping Flatland demonstrates the importance of accurate layer-dependent delay estimates and richer layer-assignment exploration [2]. Its four-dimensional placement-delay table and our six-index wire lookahead are related CAD components, not identical artifacts. Its improvements change placement guidance and quality; our experiments accelerate construction of an unchanged routing model. A future integration could combine improved placement with a faster routing-evaluation backend, and could apply the same independent-search decomposition to other precomputed delay tables after their own equivalence analysis. We have not measured that combined flow, and do not attribute its reported runtime overhead to our lookahead stage.

The practical opportunity is therefore a larger exploration budget: more architecture configurations, more placement candidates, or higher-fidelity models can be evaluated in a given time. A placement-only sweep may amortize construction

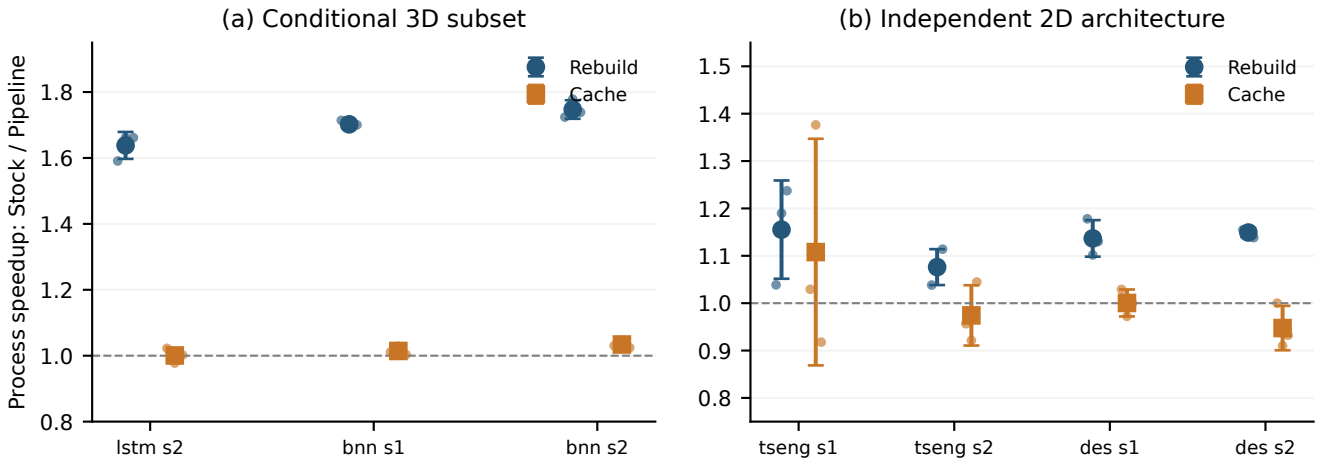


Fig. 5. Full-process paired speedups on two distinct cohorts. Markers and error bars show mean ± sample SD; small dots show all three pairs. The 3D subset was qualified in advance; k6 provides independent 2D architecture evidence. Shared-cache controls remove wire construction and support attribution, while retaining residual runtime variability.

TABLE VIII
STAGE MEMORY DIAGNOSTICS ON HOST C, ONE INSTRUMENTED OBSERVATION PER METHOD (NOT PERFORMANCE REPEATS). RSS AND LABEL CAPACITIES ARE MIB. PEAK IS THE MAXIMUM OBSERVED DURING CONSTRUCTION, NOT AN EXACT OR GUARANTEED PEAK. LABELS COUNT WIRE-FLOOD EXPANDED/VISITED ARRAYS ONLY; TABLE BOUNDS EXCLUDE VECTOR PAYLOADS, QUEUES, ALLOCATOR STATE, AND COMMON DATA.

Graph	Method	Entry RSS	Peak RSS	Growth	Label capacity	Workspaces	Sample bound	Group tables
bnn	Group	2661.6	2823.7	162.0	64.6	4	4	1
bnn	Pipeline	2661.6	2893.3	231.7	129.1	8	16	10
bnn	Flat	2661.5	2902.6	241.1	129.1	8	40	10
DLA	Group	3881.4	4119.7	238.4	122.0	4	4	1
DLA	Pipeline	3881.4	4259.8	378.4	244.0	8	16	10
DLA	Flat	3881.5	4274.6	393.0	244.0	8	40	10
clma_2d	Group	223.2	233.7	10.6	3.2	4	4	1
clma_2d	Pipeline	223.2	244.9	21.7	6.4	8	8	2
clma_2d	Flat	223.1	245.1	22.0	6.4	8	8	2
bnn_2d	Group	2196.0	2320.8	124.7	79.0	4	4	1
bnn_2d	Pipeline	2196.0	2404.8	208.8	157.9	8	16	6
bnn_2d	Flat	2196.1	2405.4	209.3	157.9	8	16	6

through caching; a topology/cost sweep is more likely to repeatedly invoke it. Distinguishing these regimes makes the value proposition concrete without assuming universal reconstruction.

B. An Execution Contract for Future Accelerators

The present work resolves a dependency problem that additional hardware alone would not solve. If a host still dispatches one four-sample group and waits at its boundary, adding more engines cannot expose the forty independent searches already available in the graph. Our decomposition supplies a global task interface and a deterministic consumer boundary that can be retained when changing the execution backend.

A future accelerator can receive descriptors $(g, j, r_{g,j})$, read shared graph/cost data, maintain private search labels, and return layer-indexed (d, q) records tagged with their original ordinal. An ordered collector then applies the same strict-delay rule and commits to the native layer-pair map. Pipeline token bounds provide backpressure at the result interface. The slice analysis identifies independent completion regions; compact cells reduce the structural cost of buffering their results. These are concrete design inputs for a replicated CPU/GPU/FPGA search engine, rather than a claim that CPU speedup transfers numerically to a device.

Several costs remain hardware-dependent: irregular RR memory access, priority queues, label initialization, graph transfer, queue/result buffering, and the precision/order needed for equivalent costs. Wire-label capacities and stage RSS quantify part of the replication cost, but not off-chip bandwidth or accelerator area. A hardware prototype must additionally demonstrate host/device transfer accounting, numerical equivalence, and a legal full-flow benefit. The CPU implementation provides a validated functional reference and task-level workload for that evaluation; no FPGA-board or GPU performance result is inferred here.

C. Limits of the Current Evidence

The method exposes available sample parallelism, not unlimited parallelism in every stage. Source/OPIN work, interpolation, ordered reduction, graph loading, and route checking

TABLE IX

REPRESENTATION/SCHEDULING ABLATION: CONTROL GROUP TIME DIVIDED BY CANDIDATE TIME, MEAN ± SAMPLE SD ($n = 3$). ALL 120 FORMAL NATIVE RUNS ARE LEGAL AND RETAIN THE REFERENCE WIRE-MAP BYTES. SLICE ALSO INCLUDES A REFACTOR; COMPACT VERSUS SLICE ISOLATES REPRESENTATION IN ONE EXECUTABLE, WHILE BOUNDED CHANGES SCHEDULING AND COMMIT LIFETIMES TOGETHER.

Circuit	Cores	Mode	Slice	Compact	Pipeline	Bounded
GEMM	4	rebuild	0.989 ± 0.007	0.993 ± 0.010	0.995 ± 0.014	0.991 ± 0.012
GEMM	4	reuse	1.008 ± 0.009	1.003 ± 0.018	1.004 ± 0.013	1.005 ± 0.018
GEMM	8	rebuild	1.005 ± 0.016	1.003 ± 0.007	1.093 ± 0.003	1.057 ± 0.046
GEMM	8	reuse	0.993 ± 0.014	0.996 ± 0.002	0.997 ± 0.004	0.995 ± 0.007
elliptic	4	rebuild	0.981 ± 0.028	0.998 ± 0.020	0.999 ± 0.021	0.984 ± 0.013
elliptic	4	reuse	0.980 ± 0.030	0.990 ± 0.027	1.006 ± 0.016	0.975 ± 0.018
elliptic	8	rebuild	0.993 ± 0.007	1.002 ± 0.027	0.998 ± 0.027	0.985 ± 0.023
elliptic	8	reuse	0.976 ± 0.082	0.965 ± 0.064	0.976 ± 0.040	0.990 ± 0.014

can become residual bottlenecks. If construction occupies fraction f of a process and accelerates by a, an unchanged-other-work model gives

$$S_{\mathrm{proc}} = \frac{1}{1 - f + f/a}. \tag{7}$$

For optimized eight-core GEMM, the construction fraction is $(10.37 \pm 0.33)\%$; halving it would yield only about $1.055\times$ in that idealized model. Further end-to-end gains require examining the remaining work, not continuing to extrapolate flood scaling.

Native 3D validation covers two layers and a limited set of vertical fabrics. More layers, different vertical resource densities, and heterogeneous sampling patterns require additional evaluation. Shared hosts and three runtime repeats limit inference about small differences; repeats do not replace independent circuits or layouts. The clma stage is short, and a preserved storage-preflight pause separates one BNN-2D stage pair. No rerun was substituted, but the interruption further limits interpretation of small stage differences. The substantive results are the large construction reductions, scaling beyond the group width, preserved native semantics, and measured benefits on qualified complete flows.

VIII. CONCLUSION

Multicore routing support does not by itself expose the parallelism available during lookahead construction. We decompose layer-aware native construction into independent floods, original-order cost reduction, and explicitly bounded execution/commit interfaces. Cross-group scheduling removes the four-sample group bottleneck and enables approximately twelvefold one-to-sixteen-core construction scaling on two large 3D graphs. At eight cores, GEMM process speedup is $1.638 \pm 0.005\times$; a qualified large 3D subset and independent 2D architectures corroborate full-flow benefit. Direct global-loop controls identify the source of the gain, while slice and storage studies expose further resource tradeoffs. The resulting execution contract provides an equivalent, measurable foundation for faster 3D architecture exploration and future hardware-parallel routing-cost computation.

REFERENCES

[1] I. Youssef, H. Yang, and C. Hao, "LaZagna: An open-source framework for flexible 3D FPGA architectural exploration," arXiv:2505.05579, 2025. [Online]. Available: https://arxiv.org/abs/2505.05579

[2] C. Hao, A. B. Kahng, B. Pramanik, and I. Youssef, "Escaping flatland: A placement flow for enabling 3D FPGAs," in *Proceedings of the 63rd ACM/IEEE Design Automation Conference (DAC)*, 2026. [Online]. Available: https://doi.org/10.1145/3770743.3804424

[3] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proceedings of the ACM International Symposium on Field-Programmable Gate Arrays*, 1995, pp. 111–117. [Online]. Available: https://janders.eecg.utoronto.ca/1387_2015/readings/pathfinder.pdf

[4] K. E. Murray, O. Petelin, S. Zhong, J. M. Wang, M. ElDafrawy, J.-P. Legault, E. Sha, A. G. Graham, J. Wu, M. J. P. Walker, H. Zeng, P. Patros, J. Luu, K. B. Kent, and V. Betz, "VTR 8: High performance CAD and customizable FPGA architecture modelling," *ACM Transactions on Reconfigurable Technology and Systems*, 2020. [Online]. Available: https://aarongraham9.github.io/vtr8_trets.pdf

[5] M. A. Elgammal, A. Mohaghegh, S. G. Shahrouz, F. Mahmoudi, F. Kosar, K. Talaei, J. Fife, D. Khadivi, K. Murray, A. Boutros, K. B. Kent, J. Goeders, and V. Betz, "VTR 9: Open-source CAD for fabric and beyond FPGA architecture exploration," *ACM Transactions on Reconfigurable Technology and Systems*, 2025. [Online]. Available: https://docs.verilogtorouting.org/en/latest/zreferences/

[6] O. Petelin and V. Betz, "The speed of diversity: Exploring complex FPGA routing topologies for the global metal layer," in *International Conference on Field Programmable Logic and Applications (FPL)*, 2016. [Online]. Available: https://www.eecg.utoronto.ca/~vaughn/papers/fpl2016_complex_routing.pdf

[7] A. Boutros, F. Mahmoudi, A. Mohaghegh, S. More, and V. Betz, "Into the third dimension: Architecture exploration tools for 3D reconfigurable acceleration devices," in *International Conference on Field-Programmable Technology (ICFPT)*, 2023, pp. 198–208. [Online]. Available: https://andrewboutros.github.io/assets/pdf/papers/026_fpt2023_3dfpga.pdf

[8] A. Poolad, S. G. Shahrouz, A. Boutros, and V. Betz, "Modeling, optimizing and exploring multi-die FPGA routing architectures," arXiv:2606.06421v3, 2026, author manuscript of FPL 2026 paper. [Online]. Available: https://arxiv.org/abs/2606.06421v3

[9] A. Singer, H. Yan, G. Zhang, M. C. Jeffrey, M. Stojilovic, and V. Betz, "MultiQueue-based FPGA routing: Relaxed A* priority ordering for improved parallelism," in *International Conference on Field-Programmable Technology (FPT)*, 2024. [Online]. Available: https://www.eecg.utoronto.ca/~mcj/papers/2024.mqrouter.fpt.pdf

[10] Verilog-to-Routing Project, "VPR command-line options," 2026, accessed November 4, 2026; experiments use a separately locked revision. [Online]. Available: https://docs.verilogtorouting.org/en/latest/vpr/command_line_usage/

[11] Intel, "Working on the assembly line: parallel_pipeline," oneTBB Developer Guide and API Reference, version 2021.9, 2023. [Online]. Available: https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-9/working-on-the-assembly-line-parallel-pipeline.html